

VLSI INTERNSHIP – TASK 6

RTL DESIGN OF PROCESSOR COMPONENTS

(CONTROL UNIT + DATAPATH INTEGRATION)

Submitted By : Likhith Gowda H R

Department of Electronics and Communication Engineering

Dayananda Sagar Academy of Technology and Management (DSATM)

Internship Domain : VLSI Design

Organization : Maincrafts Technology

September 2026

1. Introduction

A processor is a fundamental component of a digital system that performs instruction processing, data manipulation, and control operations. In a basic processor architecture, the instruction is fetched, decoded, and executed using different hardware components such as the Control Unit, Instruction Decoder, Arithmetic Logic Unit (ALU), and Datapath.

In this task, a basic processor architecture was designed and implemented using Verilog HDL. A Control Unit based on a finite state machine was developed to generate the FETCH, DECODE, and EXECUTE states. An Instruction Decoder was implemented to decode the instruction opcode and generate the corresponding ALU operation. These components were integrated with an ALU to form a simple processor datapath. The processor was functionally verified using dedicated Verilog testbenches. Icarus Verilog was used for simulation, and GTKWave was used to analyze the generated waveforms. The simulation verified instruction decoding, ALU operations, and Control Unit state transitions.

1.1 Objectives

- To understand the basic architecture of a processor.
- To design a Control Unit using a finite state machine.
- To implement FETCH, DECODE, and EXECUTE states.
- To design an Instruction Decoder using Verilog HDL.
- To decode instruction opcodes into ALU control signals.
- To integrate the Control Unit, Instruction Decoder, and ALU.
- To verify processor instruction execution using a testbench.
- To analyze simulation waveforms using GTKWave.

1.2 Tools Used

- Verilog HDL
- Icarus Verilog
- GTKWave
- Ubuntu (WSL)

2. Basic Processor Architecture

The basic processor architecture consists of an Instruction input, Instruction Decoder, Control Unit, Arithmetic Logic Unit (ALU), and Result output. The instruction is first provided to the processor and the opcode field is decoded to determine the required ALU operation. The operands are extracted from the instruction and supplied to the ALU. The Control Unit manages the processor operation through the FETCH, DECODE, and EXECUTE states.

The implemented processor follows the instruction flow:

Instruction $\rightarrow$ Decoder $\rightarrow$ ALU $\rightarrow$ Result

The Control Unit operates as an FSM and provides the basic processor sequencing through the FETCH, DECODE, and EXECUTE states

2.1 Processor Architecture

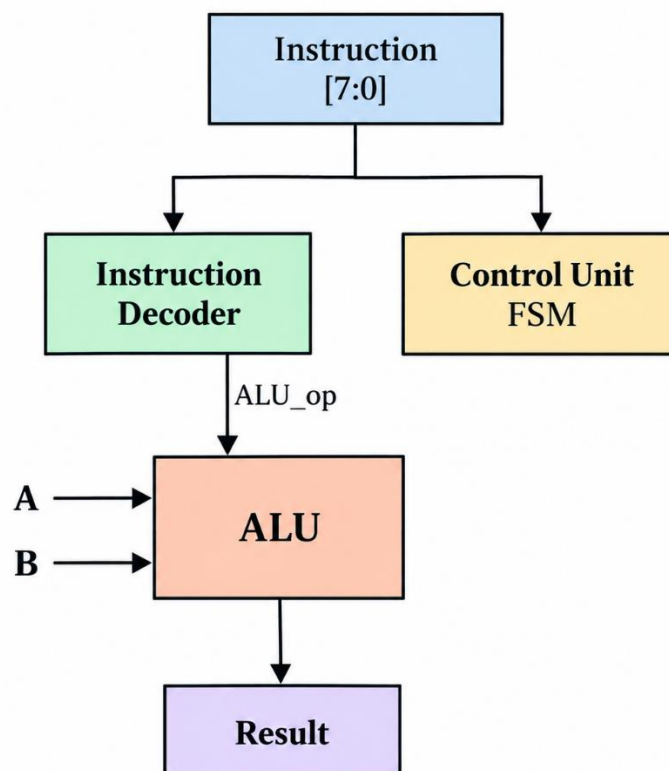


Figure 2.1: Basic Processor Architecture

The architecture illustrates the interaction between the Instruction Decoder, Control Unit, and ALU. The instruction is decoded to generate the required ALU control signal, while the Control Unit manages the processor sequence through the FETCH, DECODE, and EXECUTE states. The selected operands are processed by the ALU to generate the required result

2.2 Instruction Format

The processor uses an 8-bit instruction format. The instruction is divided into an opcode field and two operand fields. The opcode occupies bits [7:5], operand A occupies bits [4:3], operand B occupies bits [2:1], and bit [0] is unused. The opcode is supplied to the Instruction Decoder to generate the corresponding ALU operation.



Figure 2.2: 8-bit Processor Instruction Format

3. Control Unit (FSM) Implementation

The Control Unit is responsible for controlling the sequence of operations in the processor. In this task, a finite state machine (FSM) based Control Unit was implemented using Verilog HDL. The FSM consists of three states: **FETCH**, **DECODE**, and **EXECUTE**. After reset, the Control Unit starts in the FETCH state and then sequentially transitions to DECODE and EXECUTE before returning to FETCH. The state transitions are synchronized with the clock signal.

3.1 Verilog Code

control_unit.v

```
module control_unit(
    input clk,
    input reset,
    output reg [1:0] state);
parameter FETCH    = 2'b00,
              DECODE = 2'b01,
              EXECUTE = 2'b10;
always @(posedge clk or posedge reset) begin
    if (reset)
        state <= FETCH;
    else begin
        case(state)
            FETCH:  state <= DECODE;
            DECODE: state <= EXECUTE;
            EXECUTE: state <= FETCH;
            default: state <= FETCH;
        endcase
    end
end
endmodule
```

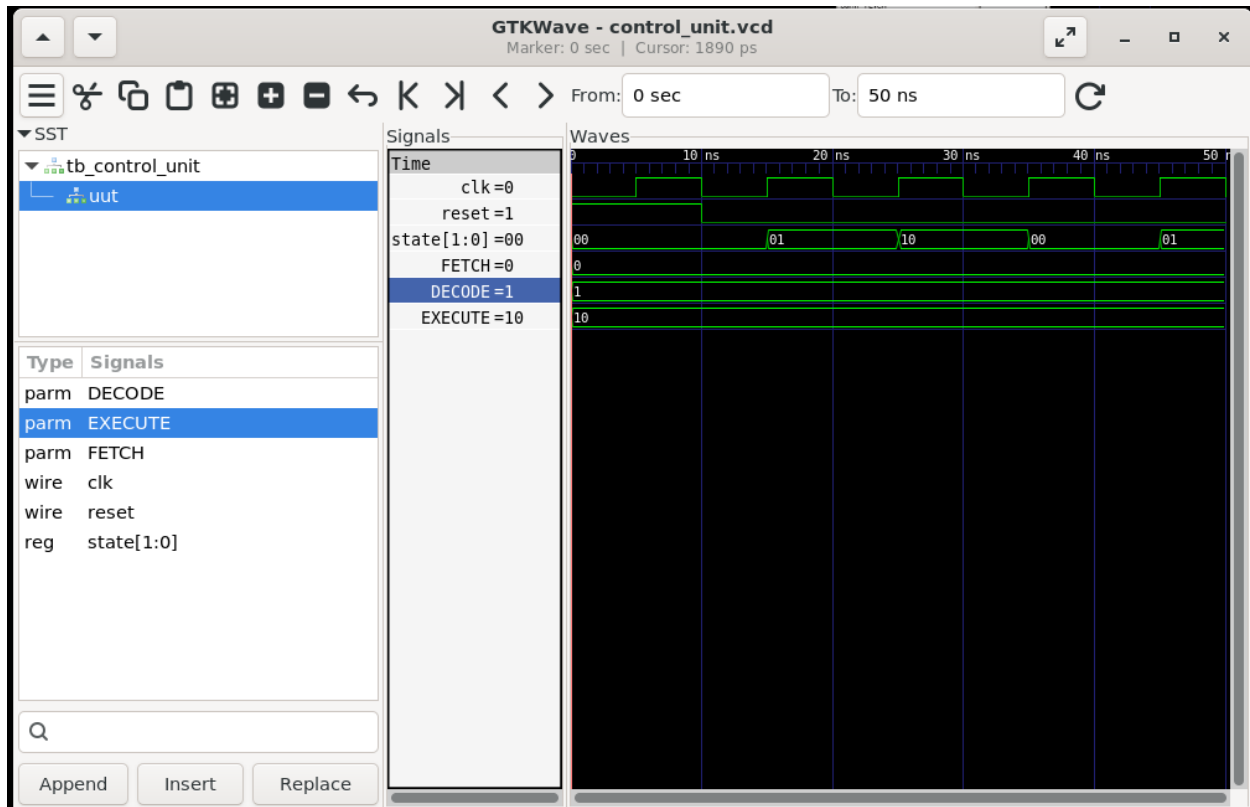
3.2 Testbench Code

The following testbench verifies the state transitions of the Control Unit by applying the clock and reset signals and observing the generated FSM states.

tb_control_unit.v

```
module control_unit(  
    input clk,  
    input reset,  
    output reg [1:0] state);  
  
parameter FETCH    = 2'b00,  
             DECODE  = 2'b01,  
             EXECUTE = 2'b10;  
  
always @(posedge clk or posedge reset) begin  
    if (reset)  
        state <= FETCH;  
    else begin  
        case(state)  
            FETCH:  state <= DECODE;  
            DECODE: state <= EXECUTE;  
            EXECUTE: state <= FETCH;  
            default: state <= FETCH;  
        endcase  
    end  
end  
  
endmodule
```

3.3 Waveform Output



Observation:

The Control Unit was successfully simulated using Icarus Verilog and GTKWave. The waveform confirms that the FSM starts from the FETCH state after reset and sequentially transitions through the DECODE and EXECUTE states before returning to FETCH. The observed state sequence validates the correct implementation of the processor control sequence.

4. Instruction Decoder Implementation

The Instruction Decoder is responsible for interpreting the opcode field of the processor instruction and generating the corresponding ALU control signal. In this implementation, a 3-bit opcode is decoded to select the required arithmetic or logical operation. The implemented instruction set includes Addition, Subtraction, AND, and OR operations.

4.1 Verilog Code

The following Verilog HDL code implements the Instruction Decoder using a combinational case statement. The decoder generates the corresponding ALU operation based on the input opcode.

decoder.v

```
module decoder(  
    input [2:0] opcode,  
    output reg [2:0] alu_op  
);  
always @(*) begin  
    case(opcode)  
        3'b000: alu_op = 3'b000; // ADD  
        3'b001: alu_op = 3'b001; // SUB  
        3'b010: alu_op = 3'b010; // AND  
        3'b011: alu_op = 3'b011; // OR  
        default: alu_op = 3'b000;  
    endcase  
end  
endmodule
```

4.2 Testbench Code

The following testbench verifies the Instruction Decoder by applying the supported opcode values and observing the corresponding ALU control signals.

tb_decoder.v

```
`timescale 1ns/1ps

module tb_decoder;

reg [2:0] opcode;

wire [2:0] alu_op;

decoder uut (

    .opcode(opcode),

    .alu_op(alu_op)

);

initial begin

    $dumpfile("decoder.vcd");

    $dumpvars(0, tb_decoder);

    opcode = 3'b000; #10;    // ADD

    opcode = 3'b001; #10;    // SUB

    opcode = 3'b010; #10;    // AND

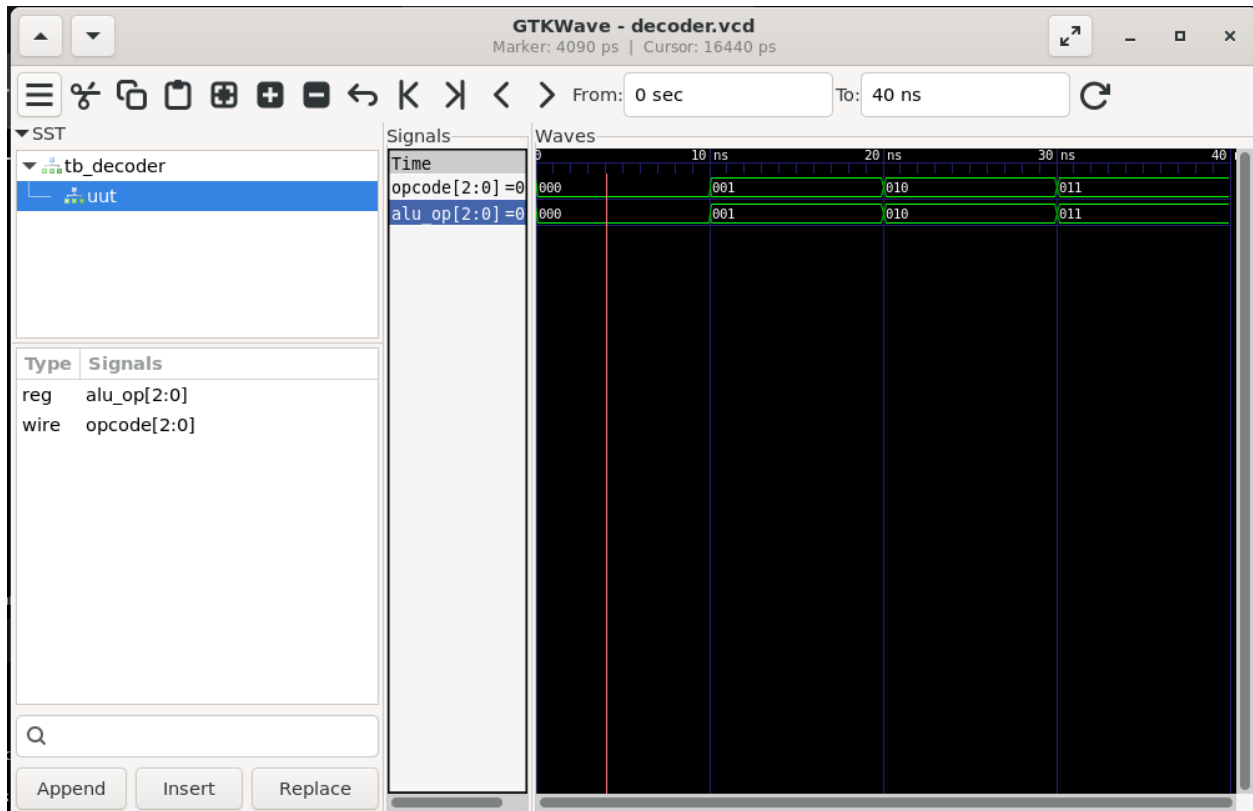
    opcode = 3'b011; #10;    // OR

    $finish;

end

endmodule
```

4.3 Waveform Output



Observation:

The Instruction Decoder was successfully simulated using Icarus Verilog and GTKWave. The waveform confirms that each opcode generates the corresponding ALU control signal. The decoder correctly identifies the ADD, SUB, AND, and OR operations, validating the instruction decoding functionality.

5. Processor Datapath Integration

The processor data path integrates the Instruction Decoder and Arithmetic Logic Unit (ALU) to perform instruction-based operations. The 8-bit instruction contains the opcode and operand fields. The opcode is decoded by the Instruction Decoder to generate the ALU control signal, while the operand fields are extracted and supplied to the ALU. The Control Unit provides the processor sequencing through the FETCH, DECODE, and EXECUTE states.

The integrated processor was implemented using Verilog HDL and verified using a dedicated testbench. The simulation demonstrates the complete flow from instruction input to opcode decoding, ALU operation, and final result generation.

5.1 Processor Architecture and Data Flow

The processor follows a simple instruction execution flow in which the instruction is decoded and the selected operation is performed by the ALU. The Control Unit operates independently as an FSM to provide the required processor state sequence.

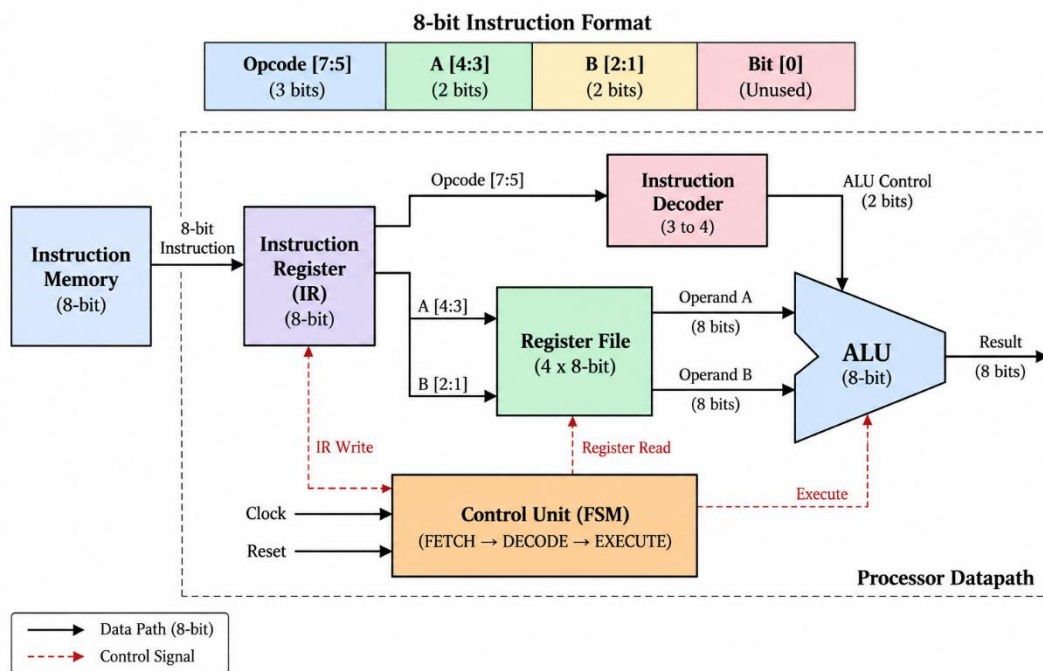


Figure 5.1: Integrated Processor Datapath

5.2 Verilog Code

The following Verilog HDL code implements the integrated processor. The processor extracts the opcode and operand fields from the 8-bit instruction, while the Instruction Decoder generates the required ALU operation. The Control Unit manages the processor state sequence, and the ALU performs the selected arithmetic or logical operation to generate the result.

processor.v

```
module alu(  
    input [3:0] A,  
    input [3:0] B,  
    input [2:0] sel,  
    output reg [3:0] Y  
);  
always @(*) begin  
    case(sel)  
        3'b000: Y = A + B;    // ADD  
        3'b001: Y = A - B;    // SUB  
        3'b010: Y = A & B;    // AND  
        3'b011: Y = A | B;    // OR  
        3'b100: Y = A ^ B;    // XOR  
        default: Y = 4'b0000;  
    endcase  
end  
endmodule  
module processor(  
    input clk,  
    input reset,  
    input [7:0] instruction,  
    output [3:0] result
```

```

);
wire [1:0] state;
wire [2:0] opcode;
wire [2:0] alu_op;
wire [3:0] A;
wire [3:0] B;

// Instruction fields
assign opcode = instruction[7:5];
assign A = {2'b00, instruction[4:3]};
assign B = {2'b00, instruction[2:1]};

// Control Unit
control_unit cu (
    .clk(clk),
    .reset(reset),
    .state(state)
);

// Instruction Decoder
decoder d1 (
    .opcode(opcode),
    .alu_op(alu_op)
);

// ALU
alu al (
    .A(A),
    .B(B),
    .sel(alu_op),
    .Y(result)
);

Endmodule

```

5.3 Testbench Code

A dedicated testbench was developed to verify the integrated processor using different instructions. ADD, SUB, AND, and OR operations were applied, and the resulting outputs were observed through simulation.

tb_processor.v

```
`timescale 1ns/1ps

module tb_processor;

    reg clk;

    reg reset;

    reg [7:0] instruction;

    wire [3:0] result;

    processor uut (

        .clk(clk),

        .reset(reset),

        .instruction(instruction),

        .result(result)

    );

    // Clock generation

    always #5 clk = ~clk;

    initial begin

        $dumpfile("processor.vcd");

        $dumpvars(0, tb_processor);

        clk = 0;

        reset = 1;
```

```
instruction = 8'b00000000;

// Apply reset

#10;

reset = 0;


// ADD: A=3, B=2

instruction = 8'b00011100;

#10;

// SUB: A=3, B=1

instruction = 8'b00111010;

#10;

// AND: A=2, B=1

instruction = 8'b01010010;

#10;

// OR: A=3, B=1

instruction = 8'b01111010;

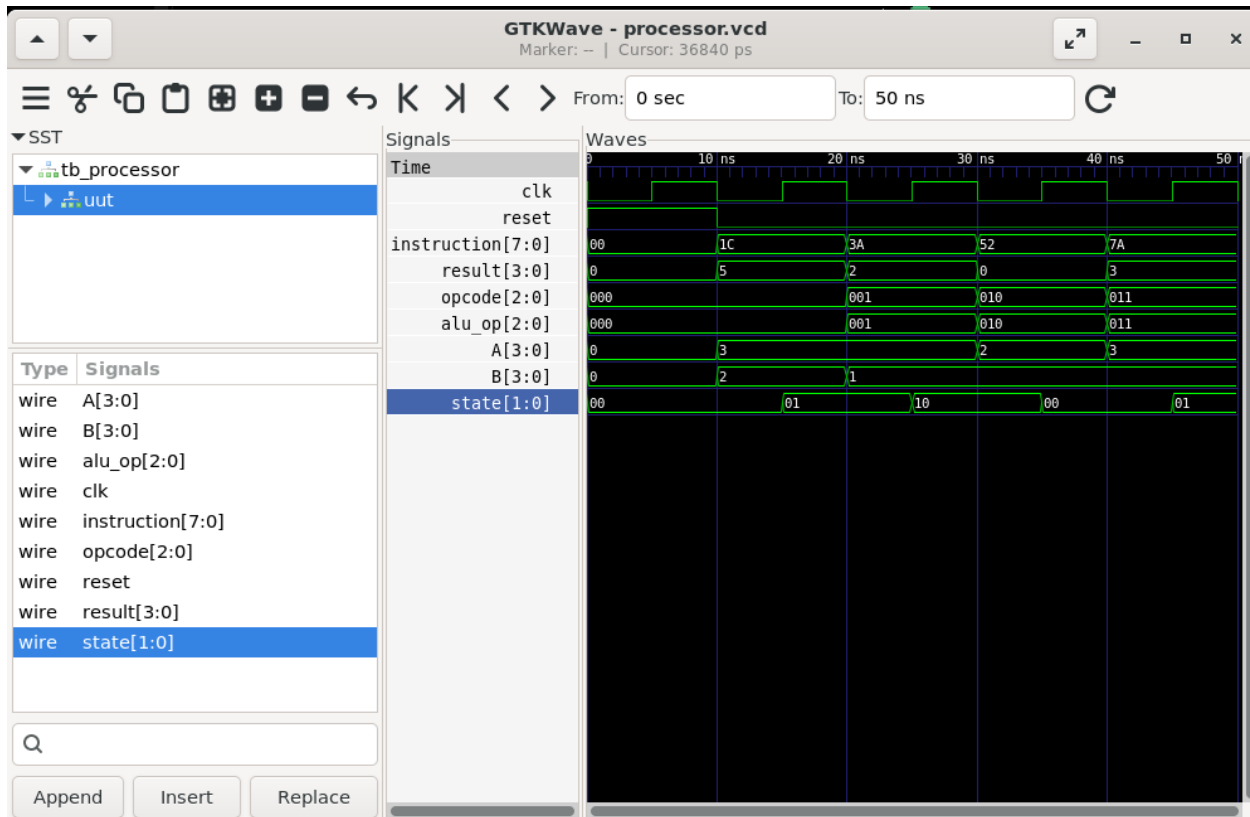
#10;

$finish;

end

endmodule
```

5.4 Waveform Output



Observation:

The processor was successfully simulated using Icarus Verilog and GTKWave. The waveform verifies the correct decoding of the instruction opcode, extraction of operands, generation of the ALU operation, and execution of the selected instruction. The processor produces the expected results for the implemented instructions: ADD = 5, SUB = 2, AND = 0, and OR = 3. The Control Unit also follows the FETCH → DECODE → EXECUTE sequence, confirming the successful integration of the processor components.

6. Results

The Control Unit, Instruction Decoder, and Processor Datapath were successfully implemented using Verilog HDL. The Control Unit correctly generated the FETCH, DECODE, and EXECUTE states. The Instruction Decoder successfully decoded the supported opcodes and generated the corresponding ALU control signals. The integrated processor correctly performed ADD, SUB, AND, and OR operations with the expected results. Functional verification was performed using Icarus Verilog, and the generated waveforms were analyzed using GTKWave. The simulation results confirmed the correct operation and integration of the processor components.

7. Applications

- Basic Processor Design
- CPU Datapath Design
- FPGA-Based Processor Implementation
- ASIC Design
- Embedded Systems
- Computer Architecture
- Digital System Design
- VLSI Design and Verification

8. Conclusion

The Control Unit, Instruction Decoder, and Processor Datapath were successfully implemented and verified using Verilog HDL. The simulation results confirmed the correct operation of the FETCH, DECODE, and EXECUTE states, instruction decoding, and ALU operations. The successful integration of these components provided practical understanding of basic processor architecture and datapath design in VLSI systems.

■ References

1. Icarus Verilog Documentation
2. GTKWave User Guide
3. Verilog HDL – Samir Palnitkar
4. Digital Design – Morris Mano
5. IEEE Verilog HDL Standard